Traduce Señas:

Manual de instalación

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

4 de octubre de 2026

Tabla de contenido

Introducción 3

1. Conceptos fundamentales 4

    1.1 Características generales (puertos) 4

    1.2 Plataforma de base de datos (migraciones) 4

    1.3 Variables de configuración (solo nombres) 5

2. Instalación 6

    2.1 Requisitos 6

    2.2 Obtener el código 6

    2.3 Instalación con Docker (ruta recomendada) 7

    2.4 Camino alternativo sin los cambios sin commit 9

    2.5 Modo desarrollo sin Docker 9

        2.5.1 Choque de puerto 3000 10

    2.6 Instalación en servidor de producción 10

3. Configuración y primer uso 10

    3.1 Comprobar que funciona 10

    3.2 Crear la primera cuenta y activarla 12

    3.3 Primer administrador 12

    3.4 Reiniciar la base de datos (borra todos los datos) 12

4. Operación 13

5. Problemas comunes 13

Referencias 15

Introducción

Este manual explica cómo poner Traduce Señas a funcionar **en un equipo local** con Docker. No cubre producción. No incluye valores de configuración: las claves y contraseñas las defines tú y **nunca** se escriben en este manual ni en el repositorio.

Advertencia importante

Hoy la instalación **completa con Docker** depende de cambios que **todavía no están en ningún commit**:

- El `Dockerfile` con `HEALTHCHECK` y el `.dockerignore` nuevos existen solo como cambios **sin commit** en el disco del Backend (rama local `chore/docker-backend`). Esa rama es **anterior** a `develop`: no tiene verificación de correo, `/api/iam`, `users.status` ni las migraciones 003-006. **No sirve para ejecutar el sistema descrito en estos manuales.** El Dockerfile commiteado es idéntico en las dos ramas, por eso los cambios sin commit se pueden llevar a `develop` (sección 2.3).
- El `Dockerfile` que sí está en `origin/develop` **no define `HEALTHCHECK`**, y el `docker-compose.yml` hace que el frontend espere a que el backend esté "sano" (`service_healthy`). Con el Dockerfile de `develop` el frontend **no arranca** con `docker compose up`.
- Los archivos `docker-compose.yml`, `build-frontend.sh` y `build-backend.sh` **no están en ninguno de los tres repositorios**: viven en la carpeta local `infra/` de la carpeta del proyecto (`sena/`). Quien clone solo los repos no los tiene; pídelos al equipo.

La sección 2.3 es la ruta recomendada y la sección 2.4 una alternativa.

1. Conceptos fundamentales

1.1 Características generales (puertos)

Tabla 1

Características generales (puertos)

Pieza	En tu equipo	Dentro de Docker
Web (nginx)	`http://127.0.0.1:8080`	puerto 80
Backend (API)	`http://127.0.0.1:3000`	puerto 3000
PostgreSQL 16	`127.0.0.1:5433`	puerto 5432

Todos los puertos se publican **solo en `127.0.0.1`**: nadie más en la red puede entrar. Para probar la app desde un celular real habría que quitar `127.0.0.1:` del puerto del backend, y hacerlo solo en redes de confianza.

La web (nginx; F5, Inc., s.f.) sirve la aplicación y reenvía todo lo que empiece por `/api/` al backend. No hay Redis ni MongoDB.

1.2 Plataforma de base de datos (migraciones)

El compose monta `../repos/Backend/migrations` (relativo a `infra/`) en `/docker-entrypoint-initdb.d`. PostgreSQL ejecuta esos archivos `.sql` en orden **solo cuando el volumen está vacío** (la primera vez; The PostgreSQL Global Development Group, s.f.).

- En `origin/develop` hay siete migraciones: `000_users.sql`, `001_translations.sql`, `002_sign_templates.sql`, `003_iam_roles_permissions.sql`, `004_analytics_usage_events.sql`, `005_auth_email_normalization_user_tokens.sql` y `006_users_status_email_verification.sql`.
- **El esquema depende de la rama que esté en disco** en `repos/Backend/` (que debe ser `develop` al día, `2c4e62d`): si no lo es, faltan las migraciones 003 a 006 y las funciones de roles, estadísticas y verificación de correo no funcionarán. La rama local `chore/docker-backend` (y la `develop` local atrasada, `ad9815d`) solo tienen las migraciones 000 a 002:

con ellas el registro falla, `/api/iam/me/access` da 404 y los SQL del primer uso (`users.status`, `user_roles`) no funcionan.

- Si cambias de rama o una migración falla, hay que **reiniciar el volumen**.

1.3 Variables de configuración (solo nombres)

El backend lee estas variables (`Backend@develop:src/shared/config/config.ts`). Se definen en `repos/Backend/.env`, que **nunca** se sube a git:

Tabla 2

Variables de configuración (solo nombres)

Variable	Para qué
`PORT`	Puerto del backend (si falta, 3000).
`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	Conexión a PostgreSQL. **Con Docker Compose, el compose las fija por ti**; no hace falta ponerlas en `.env`.
`JWT_SECRET`	Secreto para firmar las sesiones. **Obligatorio.** Elige un texto largo y aleatorio.
`EMAIL_USER`, `EMAIL_PASS`	Cuenta de correo (servicio `gmail`) para enviar los códigos de verificación.

Pasos:

1. Copia el archivo de ejemplo: `cp repos/Backend/.env.example repos/Backend/.env` (desde `sena/`).
2. Abre `repos/Backend/.env` y rellena `JWT_SECRET` con un valor tuyo.
3. Si quieres que lleguen los correos con el código, rellena `EMAIL_USER` y `EMAIL_PASS` (en Gmail suele requerirse una "contraseña de aplicación"). Si no lo haces, el registro funciona pero el código no llega: activa la cuenta a mano (sección 3.2).

Las credenciales de la base de datos del compose son valores de desarrollo definidos en el propio `docker-compose.yml`. No los copies a ningún documento ni los uses en un servidor real.

2. Instalación

2.1 Requisitos

Tabla 3

Requisitos

Necesitas	Detalle
Git	Para clonar los repositorios.
Node.js 20 o superior	Entorno de ejecución de JavaScript (OpenJS Foundation, s.f.). Solo para el modo desarrollo sin Docker (sección 2.5). Las imágenes usan Node dentro del contenedor.
Docker Desktop	Con `docker` y `docker compose` disponibles (Docker Inc., s.f.).
Espacio en disco	Reserva varios GB libres: imágenes de Docker, `node_modules` y el modelo de reconocimiento.
Internet	Para descargar imágenes y dependencias, y para que el navegador baje MediaPipe al usar la cámara.
Una terminal tipo bash	En Windows, Git Bash. Los scripts `build-*.sh` son bash.
Una cuenta de correo para enviar códigos	Opcional en pruebas (ver sección 1.3). El envío usa el servicio `gmail` de `nodemailer`.

2.2 Obtener el código

Esta es la disposición **recomendada** (la usada en la máquina del equipo de desarrollo): una carpeta `sena/` con los tres repositorios dentro de `repos/` y los archivos de infraestructura en `infra/`. `sena/` no es un repositorio git. Tus compañeros pueden tener otra disposición: el compose y los scripts están fuera de los repos y solo necesitan encontrar `Backend` y `Frontend` por rutas relativas (el compose usa `../repos/...` desde `infra/`; si cambias la disposición, ajusta esas rutas).

```
sena/
  repos/
    Backend/    <- github.com/mauro-1508/backendTS      (rama develop)
    Frontend/   <- github.com/tadeo77789/frontendTS     (rama develop)
    Docs/       <- github.com/code-sena/trans-sl-docs   (rama main)
  infra/
    docker-compose.yml    <- NO está en ningún repo; pídelo al equipo
    build-frontend.sh     <- idem
    build-backend.sh      <- idem

mkdir -p sena/repos sena/infra && cd sena
git clone -b develop https://github.com/mauro-1508/backendTS.git
repos/Backend
git clone -b develop https://github.com/tadeo77789/frontendTS.git
repos/Frontend
git clone -b main https://github.com/code-sena/trans-sl-docs.git
repos/Docs
```

Los comandos de este manual se ejecutan **desde la raíz `sena/`** (los de git y `cp`) o **desde `sena/infra/`** (los de `docker compose` y los scripts `build-*.sh`). Cada bloque indica desde dónde.

En el repositorio Backend, la rama `main` solo tiene un README. El código está en `develop`: verifica con `git -C repos/Backend branch --show-current`.

Carpetas sincronizadas en la nube (OneDrive y similares). Si pones `sena/` en una carpeta sincronizada, Docker (BuildKit) no puede leer algunos de sus archivos. Por eso existen los scripts `build-*.sh` (sección 2.3). Además, marca `repos/Backend/migrations` como "Mantener siempre en este dispositivo"; si no, el montaje de las migraciones puede verse vacío.

2.3 Instalación con Docker (ruta recomendada)

Requisitos: `docker-compose.yml` y `build-*.sh` en `infra/`, y el Backend **en la rama `develop` actualizada** (commit `2c4e62d`), conservando los cambios sin commit de `Dockerfile` y `.dockerignore`.

Paso A. Pasa el Backend a `develop` y actualízalo. La `develop` local puede estar atrasada respecto a `origin/develop`:

(desde `sena/`)

```
git -C repos/Backend checkout develop
git -C repos/Backend pull --ff-only origin develop
git -C repos/Backend rev-parse --short HEAD     # debe mostrar 2c4e62d
```

Los cambios sin commit de `Dockerfile` y `.dockerignore` se conservan porque la versión commiteada de esos archivos es la misma en `chore/docker-backend` y en `develop`. Si Git se niega por otros archivos modificados (por ejemplo `.gitignore`, que sí difiere entre las ramas), guarda una copia de ese archivo, descarta tus cambios en él y repite el comando. Después del `pull`, vuelve a añadir al final de `.gitignore` las líneas de tu copia que falten (las de `liquibase.properties`): protegen archivos con credenciales y no deben perderse.

Paso B. Construye y arranca, desde `sena/infra/`:

```
cd infra
./build-frontend.sh
./build-backend.sh --dockerfile-from develop
docker compose up -d
```

Qué hace cada orden:

1. `./build-frontend.sh` construye la imagen `traduce-senas-frontend:latest`. Envía los archivos de `repos/Frontend/` a Docker como un `tar` (sin `node_modules`, `.git`, `dist` ni `.expo`) y compila la web con Expo (Expo, s.f.), dentro de la imagen. Tarda varios minutos la primera vez.
2. `./build-backend.sh --dockerfile-from develop` construye `traduce-senas-backend:latest`. Con esa opción, la rama actual coincide con la "rama del Dockerfile" y el script empaqueta directamente la carpeta de trabajo, es decir, **con el Dockerfile con `HEALTHCHECK` sin commit**. Sin la opción, el script busca el Dockerfile en `chore/docker-backend` y, estando en `develop`, aborta porque ese Dockerfile commiteado es igual al de `develop`.
    - El script **siempre mostrará** "AVISO: se esperaba feat/email-verification…": esa rama ya no existe y el aviso es inofensivo.
    - También avisa si hay cambios sin commit (se incluyen en la imagen, es lo esperado) y si falta `repos/Backend/.env`.

3. `docker compose up -d` levanta PostgreSQL, el backend y la web. El frontend espera a que el backend esté sano.

Por qué hay scripts y no `docker compose build`: la carpeta del proyecto suele estar sincronizada en la nube y BuildKit no puede leer esos archivos (reparse points). Los scripts evitan el problema enviando el contexto como `tar`.

Si cambias el código, vuelve a ejecutar el script que corresponda y luego `docker compose up -d`.

2.4 Camino alternativo sin los cambios sin commit

Si tienes solo los tres repos clonados (Backend en `origin/develop`) y el `docker-compose.yml`, sigue estos pasos:

1. Construye las imágenes con los nombres que usa el compose:

```
docker build -t traduce-senas-backend:latest repos/Backend
docker build -t traduce-senas-frontend:latest repos/Frontend
```

   (Estos dos comandos se ejecutan desde `sena/`. Si `sena/` está en una carpeta sincronizada en la nube, este paso puede fallar por lo explicado antes; mueve el proyecto a una carpeta local, p. ej. `C:\dev\sena`. Los comandos `docker compose` siguientes se ejecutan desde `sena/infra/`.)

2. Levanta primero la base y el backend: `docker compose up -d postgres backend`.
3. Levanta la web sin esperar al "healthcheck" que el Dockerfile de `origin/develop` no define: `docker compose up -d --no-deps frontend`.

Por esta ruta el backend **no tendrá estado `healthy`** (no hay `HEALTHCHECK`): en 3.1 comprueba solo que esté `running` y que la API responda.

2.5 Modo desarrollo sin Docker

Sirve para programar con recarga automática. Necesitas Node 20+ y una base PostgreSQL.

1. **Base de datos:** puedes usar la del compose: `docker compose up -d postgres` (desde `infra/`; queda en `127.0.0.1:5433`).

2. **Backend:** en `repos/Backend/`:

```
npm install
cp .env.example .env
npm run dev
```

   En `.env` pon `DB_HOST` (127.0.0.1), `DB_PORT` (5433), `DB_NAME` (traduce_senas), `DB_USER`, `DB_PASSWORD` (los del compose), `JWT_SECRET`, y si quieres `EMAIL_USER`/`EMAIL_PASS`. Otros comandos: `npm run build`, `npm start`, `npm test` (usa vitest).

3. **Frontend:** en `repos/Frontend/`: `npm install` y luego `npm run web` (Expo). La terminal te indica la dirección. En desarrollo la web llama a `http://localhost:3000/api`. Otros comandos: `npm start`, `npm run android`, `npm run ios`, `npm run typecheck`.

2.5.1 Choque de puerto 3000

El backend en desarrollo usa el puerto **3000**, el mismo que publica el contenedor `backend`. **Si ambos corren a la vez, chocan.** Antes de `npm run dev` detén el contenedor:

```
docker compose stop backend
```

(La web en Docker usa el 8080, así que con el backend de desarrollo en 3000 puedes mantener la base de Docker.)

2.6 Instalación en servidor de producción

Esta versión del proyecto no incluye un procedimiento de instalación en un servidor de producción: el sistema se ejecuta en un equipo local, como se describe en las secciones anteriores.

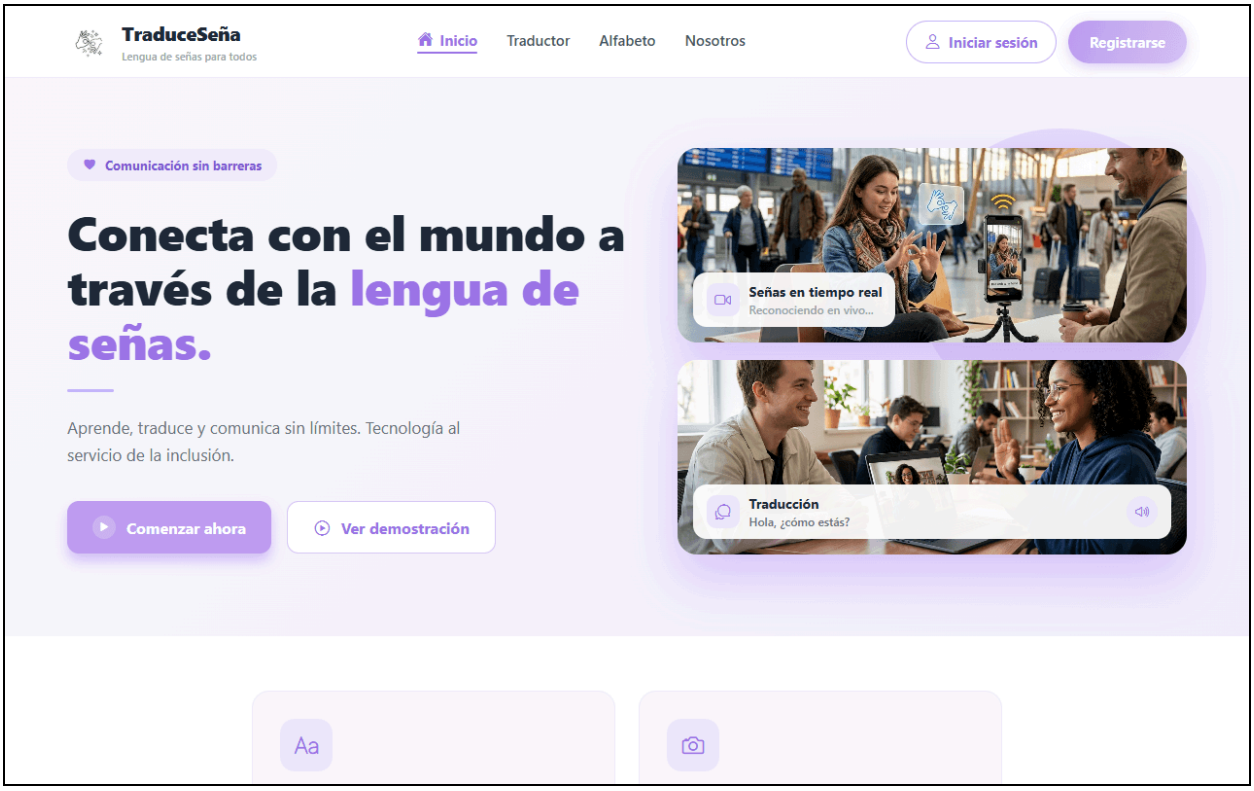
3. Configuración y primer uso

3.1 Comprobar que funciona

1. Estado de los servicios: `docker compose ps`. Los tres deben estar en marcha (`running`). Con la ruta de la sección 2.3 el backend y la base aparecen además como `healthy`; con la de la sección 2.4 el backend no.
2. Abre `http://127.0.0.1:8080`: debe cargar la página de inicio.

Figura 1

Página de inicio de la aplicación



3. Comprueba que la API responde a través de la web: `curl http://127.0.0.1:8080/api/sign-templates` debe devolver una respuesta JSON (puede venir con la lista vacía).
4. Si algo falla: `docker compose logs backend` y `docker compose logs postgres`.

3.2 Crear la primera cuenta y activarla

1. En la web, pulsa **Registrarse** y completa el formulario **Crea tu Cuenta**.
2. Las cuentas nuevas quedan **inactivas** hasta confirmar el correo con un código. Si configuraste `EMAIL_USER` y `EMAIL_PASS`, el código llega al correo. Si no, activa la cuenta a mano:

```
docker compose exec postgres psql -U <usuario_de_la_base> -d
traduce_senas -c "UPDATE users SET status='ACTIVE' WHERE
email='<correo>'"
```

(`<usuario_de_la_base>` es el usuario de PostgreSQL que define el compose.)

3.3 Primer administrador

Se hace con una orden SQL una sola vez. Está explicada paso a paso en el Manual del administrador, sección 3. En resumen, se inserta el rol `ADMIN` para ese correo en la tabla `user_roles`.

3.4 Reiniciar la base de datos (borra todos los datos)

Esto borra TODOS los datos de la base (cuentas, historial, plantillas). Haz antes una copia si te importan: `docker compose exec postgres pg_dump -U <usuario_de_la_base> traduce_senas > copia.sql`

```
docker compose down -v
docker compose up -d
```

4. Operación

Tabla 4

Operación

Quiero...	Orden
Detener todo sin borrar datos	`docker compose stop`
Volver a encender	`docker compose up -d`
Detener y quitar contenedores (los datos se conservan)	`docker compose down`
Ver mensajes del backend	`docker compose logs -f backend`
Borrar también los datos	`docker compose down -v` (**borra los datos**)
Actualizar el código	`git pull` en cada repo (`repos/Backend`, `repos/Frontend`, `repos/Docs`), reconstruir imágenes (sección 2.3) y `docker compose up -d`. Las migraciones nuevas no se aplican solas a una base existente.

5. Problemas comunes

Tabla 5

Problemas comunes

Síntoma	Causa probable	Qué hacer
`./build-backend.sh` dice que la rama no existe o que "no tiene commiteado el Dockerfile nuevo"	Usaste el script sin `--dockerfile-from develop`: la rama `chore/docker-backend` no existe o su Dockerfile no está commiteado	Ejecuta `./build-backend.sh --dockerfile-from develop` (sección 2.3) o usa la sección 2.4
`docker compose up` falla con el frontend "has no healthcheck"	El Dockerfile del backend es el de `develop`, sin `HEALTHCHECK`	Sección 2.4, paso 3
El frontend no arranca (nginx: `host not found in upstream "backend"`)	Nginx necesita que el servicio `backend` ya exista	Levanta antes `backend`; mira `docker compose logs frontend`

Falla la construcción con errores de archivos ilegibles	Carpeta sincronizada en la nube	Usa los scripts `build-*.sh` o mueve el proyecto a una carpeta local
Las tablas no existen o faltan roles/estadísticas	Volumen creado con otra rama o migración fallida	Secciones 1.2 y 3.4: reiniciar el volumen con la rama correcta
Error de base de datos tras cambiar de nombre de BD	El volumen viejo tiene otra base	`docker compose down -v` (borra datos)
No llega el código de verificación	`EMAIL_USER`/`EMAIL_PASS` sin configurar o rechazados	Activa la cuenta a mano (3.2) o configura el correo
Todos los usuarios quedan sin sesión tras reiniciar	`JWT_SECRET` cambió o está vacío	Define un `JWT_SECRET` fijo en `repos/Backend/.env`
El puerto 8080, 3000 o 5433 está ocupado	Otro programa lo usa	Cierra ese programa o cambia el puerto en el compose
La cámara no funciona en la web	El navegador no tiene permiso	Acepta el permiso. Algunos navegadores solo permiten la cámara en `localhost`/`127.0.0.1` o con HTTPS
`npm run dev` falla con "EADDRINUSE"	Puerto 3000 ocupado por el contenedor	`docker compose stop backend`

Referencias

Docker Inc. (s.f.). *Docker Compose*. Docker Docs. Recuperado el 7 de octubre de 2026, de https://docs.docker.com/compose/

Expo. (s.f.). *Expo documentation*. Recuperado el 7 de octubre de 2026, de https://docs.expo.dev/

F5, Inc. (s.f.). *nginx documentation*. Recuperado el 7 de octubre de 2026, de https://nginx.org/en/docs/

OpenJS Foundation. (s.f.). *Node.js documentation*. Recuperado el 7 de octubre de 2026, de https://nodejs.org/docs/latest/api/

The PostgreSQL Global Development Group. (s.f.). *PostgreSQL 16 documentation*. Recuperado el 7 de octubre de 2026, de https://www.postgresql.org/docs/16/